Contents

1. Objective . . . 3

2. Reference Tools . . . 3

3. Approach . . . 3

4. Architecture . . . 3

4.1 Request flow and container layout . . . 4
4.2 Project files . . . 4
4.3 The deep-learning API . . . 4
4.4 Multi-stage Dockerfile . . . 5
4.5 Optimization practices . . . 5
4.6 Image size comparison . . . 6
4.7 Running the project . . . 6
4.8 Testing and issues resolved . . . 6

5. Source Code . . . 6

5.1 app.py - Flask API . . . 7
5.2 train_model.py - model training . . . 9
5.3 test_api.py - API tests . . . 10
5.4 Dockerfile - optimized multi-stage image . . . 12
5.5 Dockerfile.baseline - un-optimized reference . . . 12
5.6 .dockerignore . . . 12
5.7 requirements.txt . . . 13
5.8 task10_docker.py - Docker workflow runner . . . 13

6. Command Execution Log . . . 15

6.1 Verify Docker (Steps 1-2) . . . 15
6.2 Build the Images and Compare Their Size (Steps 3-8) . . . 15
6.3 Run the Container Locally (Steps 9-11) . . . 17
6.4 Verify API Functionality (Steps 12-16) . . . 18
6.5 Inspect the Container and Check the Optimizations (Steps 17-23) . . . 19
6.6 Compare with the Baseline Container (Steps 24-27) . . . 21
6.7 Cleanup (Step 28) . . . 22

7. Screenshots and Observations . . . 23

7.1 Building the Optimized Image . . . 23
7.2 Image Size: Optimized versus Baseline . . . 24
7.3 Starting the Container and Health Status . . . 24
7.4 Health Check and Model Information . . . 24
7.5 Prediction and Error Handling . . . 24
7.6 API Test Suite . . . 25
7.7 The API in a Web Browser . . . 25
7.8 Non-root User and Resource Usage . . . 25
7.9 Docker Desktop - Containers Tab . . . 26

8. Results . . . 27

8.1 Results by approach area . . . 27
8.2 Optimized image compared with the baseline . . . 28
8.3 API test results (api_test_results.json) . . . 28
8.4 Sample predictions . . . 29

9. Observations . . . 29

10. Known Limitations . . . 30

11. Conclusion . . . 30

1. Objective

To package a Flask-based deep learning API into a Docker container for deployment. The deliverables are the Dockerfile (Section 5.4), the Docker image `flask-dl-api:1.0` (Sections 6 and 8) and this deployment report.

2. Reference Tools

The API is a Flask application served by **gunicorn**. The deep-learning model is a small neural network trained from scratch in NumPy; only NumPy is needed at inference time. **Docker Desktop** (v4.94.0 on macOS) and the **Docker CLI** (29.8.2) build and run the container, and a Python script records the command, output and exit code of every step.

Item	Value from the recorded run
Docker Desktop / Engine	v4.94.0 / 29.8.2 (client and server), storage driver overlayfs
Base images	python:3.12-slim (optimized, Python 3.12.15, Debian arm64) and python:3.12 (baseline)
Web framework / server	Flask 3.1.3 with Werkzeug 3.1.9 / gunicorn 26.2.0 (2 workers)
Model runtime	NumPy (pinned in requirements.txt); scikit-learn only for training
Port mapping	host 8000 to container 5000 (macOS uses host port 5000 for AirPlay)
Image architecture	arm64 (Apple Silicon)

3. Approach

The five approach steps of the task were carried out in order. Section and figure numbers refer to this report.

1. **Create a Dockerfile** - a multi-stage `Dockerfile` for the optimized image and a `Dockerfile.baseline` as un-optimized reference (Section 5.4 and 5.5).
2. **Build a Docker image** - `flask-dl-api:1.0` and `flask-dl-api:baseline` were built with `docker build` (steps 3-4, Figures 1-2).
3. **Run the container locally** - `docker run -d -p 8000:5000`, waiting until the health check reports healthy (steps 9-11, Figure 3).
4. **Verify API functionality** - curl calls, the test suite `test_api.py` (13 checks), inspection of the container and a restart test (steps 12-23, Figures 4-9).
5. **Optimize image size** - slim base image, multi-stage build, no pip cache, no training dependencies, `.dockerignore`, non-root user; the result was compared with the baseline (steps 5-8 and 24-27, Figure 2).

4. Architecture

The API runs inside one container. A client on the Mac sends HTTP requests to host port 8000, which Docker forwards to port 5000 of the container. There gunicorn passes them to the Flask application, which evaluates the NumPy model and returns JSON. Docker's HEALTHCHECK calls the `/health` route every 10 seconds.

4.1 Request flow and container layout

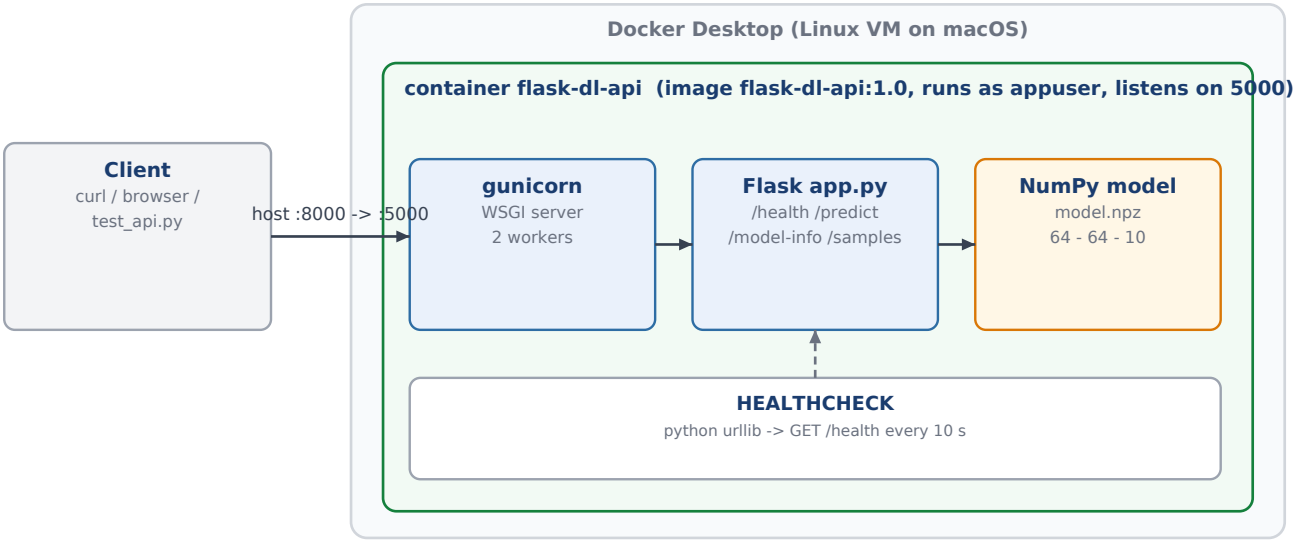


Diagram 1: Request flow from the client to the model and the built-in health check.

4.2 Project files

File	Role
`app.py`	Flask API: routes /, /health, /model-info, /samples and POST /predict with input validation and JSON errors.
`train_model.py`	Trains the neural network in NumPy on the UCI digits dataset and writes `model/model.npz`.
`model/model.npz`	Weights, metadata and ten sample images (25,458 bytes).
`requirements.txt`	Pinned runtime dependencies (Flask, Werkzeug, gunicorn, NumPy and their dependencies).
`Dockerfile`	Optimized multi-stage image flask-dl-api:1.0.
`Dockerfile.baseline`	Un-optimized reference image used for the size comparison.
`.dockerignore`	Keeps logs, reports, tests and training code out of the build context.
`test_api.py`	Black-box tests of the running API (standard library only).
`task10_docker.py`	Runs the 28 Docker steps and records the real output.

4.3 The deep-learning API

The model is a fully connected network with 64 inputs, a hidden layer of 64 ReLU units and 10 softmax outputs (64x64 + 64 + 64x10 + 10 = 4,810 parameters). It was trained from scratch in NumPy with Adam for 80 epochs on 1,437 images of the UCI handwritten-digits dataset (8x8 grayscale, pixel values 0 to 16) and reaches 97.22 % accuracy on 360 held-out images (training accuracy 100 %). The weights are loaded once when the server starts.

The routes of the API:

Method	Route	Purpose
GET	`/`	service description and list of routes
GET	`/health`	liveness check used by the Docker HEALTHCHECK
GET	`/model-info`	architecture, parameters, accuracy
GET	`/samples`	one held-out example per digit with its true label
POST	`/predict`	body {"pixels": [64 numbers 0-16]} or {"instances": [[...], ...]} (at most 100); returns digit, confidence and probabilities

Invalid requests (wrong length, non-numeric values, values outside 0-16, invalid JSON) receive a JSON error with HTTP status 400; unknown routes and wrong methods receive JSON 404 and 405.

4.4 Multi-stage Dockerfile

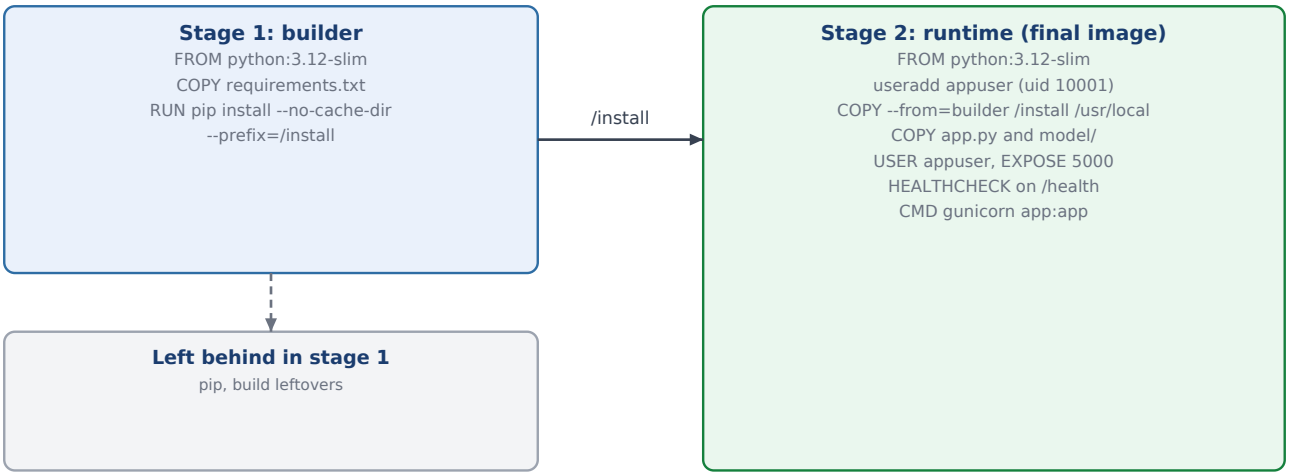


Diagram 2: The two stages of the optimized Dockerfile. Only the installed packages are copied from stage 1 into the final image.

4.5 Optimization practices

Practice	Effect and evidence in this run
python:3.12-slim instead of python:3.12	Much smaller base layers; together with the other measures the image is 303.3 MB instead of 1,753.0 MB (step 7).
Multi-stage build	Dependencies are installed in a builder stage; the final stage receives only the installed packages (78.3 MB layer, step 8).
pip install --no-cache-dir	No pip download cache is stored in a layer.
Pinned requirements.txt, copied before the code	Reproducible builds and a cached dependency layer when only app.py changes.
NumPy-only inference	scikit-learn is not in the image (step 19).
.dockerignore	Build context of only 295 B and 125 B (Figure 1); no logs, reports or training code in /app (step 20).
Non-root user appuser (uid 10001)	The process runs as uid 10001; the baseline runs as root (steps 17 and 26).
HEALTHCHECK on /health	Docker reports (healthy) about 6 seconds after the start (steps 10-11).
gunicorn instead of the Flask development server	Production WSGI server with 2 workers (22 processes in docker stats, step 22).
PYTHONDONTWRITEBYTECODE and PYTHONUNBUFFERED	No .pyc files are written and log lines appear immediately in docker logs.

4.6 Image size comparison

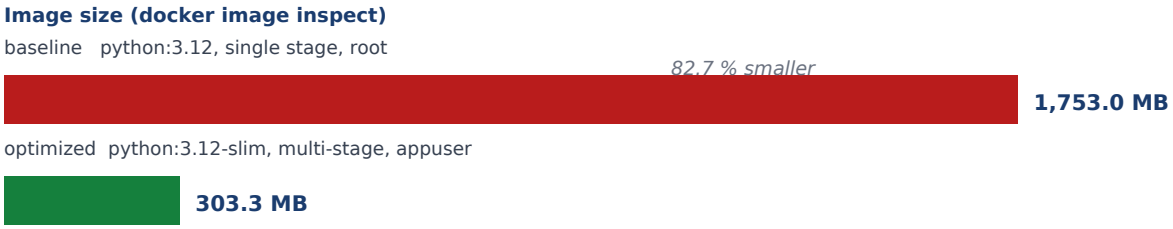


Diagram 3: Image sizes measured with docker image inspect (steps 6 and 7).

4.7 Running the project

- Install Docker Desktop, start it and wait until it shows *Engine running*.
- In the project folder run `python3 task10_docker.py`. It builds both images, starts the container, tests the API, inspects the container, compares with the baseline and removes the containers; the log is written to `docker_command_log.txt` and `docker_results.json`.
- Manually: `docker build -t flask-dl-api:1.0 .`, then `docker run -d --name flask-dl-api -p 8000:5000 flask-dl-api:1.0` and `curl http://localhost:8000/health`.

4.8 Testing and issues resolved

Issue found	Cause	Outcome
Host port 5000 cannot be used on the Mac	macOS reserves port 5000 for AirPlay Receiver.	The container listens on 5000 and is published on host port 8000 (and 8001 for the baseline); all calls work.
First attempt of the runner script failed in the container steps	18 of 28 steps ended with 'No such container: flask-dl-api' (10 of 28 succeeded). The cause was not recorded.	The script was run again; all 28 steps succeeded. This report uses the successful run.
Progress text labelled [stderr] in successful steps (3, 4)	BuildKit writes build progress to stderr.	Exit code 0 in both steps; no action needed.
Build steps show CACHED	Layers were reused from earlier builds.	Build durations are not compared; the size comparison does not depend on them.
IDs in screenshots differ from the log	The screenshots were taken from a second manual run after the script had removed its containers (see Section 7).	Container and image IDs differ; sizes and results are identical.

5. Source Code

All files needed to build and run the project are listed. Lines longer than 100 characters are wrapped and continued with the symbol ↳. Line numbers refer to the real files.

5.1 app.py - Flask API

```
"""
app.py - Flask deep-learning API (digit classifier).

Endpoints
  GET  /            service description and list of endpoints
  GET  /health      liveness check (used by the Docker HEALTHCHECK)
  GET  /model-info  model metadata (architecture, parameters, test accuracy)
  GET  /samples     one held-out example per digit (pixels + true label) for quick testing
  POST /predict     JSON {"pixels": [64 numbers]} or {"instances": [[64 numbers], ...]}

Run locally:   python app.py                (Flask development server)
Production:    gunicorn --bind 0.0.0.0:5000 app:app
"""
import json
import os
import time

import numpy as np
from flask import Flask, jsonify, request

BASE = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.environ.get("MODEL_PATH", os.path.join(BASE, "model", "model.npz"))
MAX_BATCH = 100
N_FEATURES = 64

app = Flask(__name__)

_m = np.load(MODEL_PATH, allow_pickle=False)
W1, b1, W2, b2 = (_m[k].astype(np.float32) for k in ("W1", "b1", "W2", "b2"))
SAMPLES_X, SAMPLES_Y = _m["samples_x"], _m["samples_y"]
META = json.loads(str(_m["meta"]))
STARTED = time.time()


def predict_proba(x):
    """x: array (n, 64) with pixel values 0-16 -> softmax probabilities (n, 10)."""
    x = np.asarray(x, dtype=np.float32) / 16.0
    h = np.maximum(0.0, x @ W1 + b1)
    z = h @ W2 + b2
    e = np.exp(z - z.max(axis=1, keepdims=True))
    return e / e.sum(axis=1, keepdims=True)


def bad_request(message, status=400):
    return jsonify({"error": message}), status


@app.get("/")
def index():
    return jsonify({
        "service": "Flask deep-learning API (digit classifier)",
        "model": META["name"],
        "endpoints": {"GET /health": "liveness check", "GET /model-info": "model metadata",
                      "GET /samples": "example inputs with true labels",
                      "POST /predict": 'body: {"pixels": [64 numbers 0-16]} or {"instances": [[...],
↳     ...]}'},
    })


@app.get("/health")
def health():
    return jsonify({"status": "ok", "uptime_seconds": round(time.time() - STARTED, 1)})


```

```
@app.get("/model-info")
def model_info():
    return jsonify(META)


@app.get("/samples")
def samples():
    return jsonify([{"label": int(y), "pixels": [float(v) for v in x]} for x, y in zip(SAMPLES_X,
    SAMPLES_Y)])


@app.post("/predict")
def predict():
    data = request.get_json(silent=True)
    if not isinstance(data, dict):
        return bad_request('Request body must be a JSON object with "pixels" or "instances".')
    if "pixels" in data:
        rows = [data["pixels"]]
    elif "instances" in data:
        rows = data["instances"]
        if not isinstance(rows, list) or not rows:
            return bad_request('"instances" must be a non-empty list of 64-value lists.')
        if len(rows) > MAX_BATCH:
            return bad_request(f"At most {MAX_BATCH} instances per request.")
    else:
        return bad_request('Provide "pixels" (one image) or "instances" (list of images).')
    try:
        x = np.array(rows, dtype=np.float64)
    except (ValueError, TypeError):
        return bad_request("All pixel values must be numbers.")
    if x.ndim != 2 or x.shape[1] != N_FEATURES:
        return bad_request(f"Each image needs exactly {N_FEATURES} pixel values (flattened 8x8).")
    if not np.isfinite(x).all() or x.min() < 0 or x.max() > 16:
        return bad_request("Pixel values must be finite numbers between 0 and 16.")

    start = time.perf_counter()
    probs = predict_proba(x)
    elapsed = (time.perf_counter() - start) * 1000
    results = [{"digit": int(p.argmax()), "confidence": round(float(p.max()), 4),
                "probabilities": [round(float(v), 4) for v in p]} for p in probs]
    body = {"predictions": results, "count": len(results), "inference_ms": round(elapsed, 3)}
    if "pixels" in data:
        body.update(results[0])
    return jsonify(body)


@app.errorhandler(404)
def not_found(_):
    return jsonify({"error": "Route not found."}), 404


@app.errorhandler(405)
def not_allowed(_):
    return jsonify({"error": "Method not allowed for this route."}), 405


@app.errorhandler(500)
def server_error(_):
    return jsonify({"error": "Internal server error."}), 500


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT", 5000)))
```

5.2 train_model.py - model training

```
#!/usr/bin/env python3
"""
train_model.py - trains the deep-learning model served by the Flask API.

A small fully connected neural network (64 -> 64 ReLU -> 10 softmax) is trained from
scratch in NumPy on the UCI handwritten-digits dataset (8x8 images, 1,797 samples) and
exported to model/model.npz. The API only needs NumPy at inference time, which keeps the
Docker image small. scikit-learn is used here only to load the dataset (training-time
dependency, listed in requirements-train.txt and NOT installed in the image).

Usage:  python train_model.py
"""
import json
import os
from datetime import date

import numpy as np
from sklearn.datasets import load_digits

SEED, EPOCHS, BATCH, LR, HIDDEN = 42, 80, 32, 0.002, 64
OUT = os.path.join(os.path.dirname(os.path.abspath(__file__)), "model", "model.npz")


def softmax(z):
    e = np.exp(z - z.max(axis=1, keepdims=True))
    return e / e.sum(axis=1, keepdims=True)


def forward(x, p):
    h = np.maximum(0, x @ p["W1"] + p["b1"])
    return h, softmax(h @ p["W2"] + p["b2"])


def main():
    d = load_digits()
    X, y = d.data.astype(np.float64) / 16.0, d.target
    rng = np.random.default_rng(SEED)
    idx = rng.permutation(len(X))
    cut = int(len(X) * 0.8)
    tr, te = idx[:cut], idx[cut:]
    Xtr, ytr, Xte, yte = X[tr], y[tr], X[te], y[te]

    p = {"W1": rng.normal(0, np.sqrt(2 / 64), (64, HIDDEN)), "b1": np.zeros(HIDDEN),
         "W2": rng.normal(0, np.sqrt(2 / HIDDEN), (HIDDEN, 10)), "b2": np.zeros(10)}
    m = {k: np.zeros_like(v) for k, v in p.items()}
    v = {k: np.zeros_like(val) for k, val in p.items()}
    t = 0
    for ep in range(1, EPOCHS + 1):
        order = rng.permutation(len(Xtr))
        for i in range(0, len(order), BATCH):
            b = order[i:i + BATCH]
            xb, yb = Xtr[b], ytr[b]
            h, pr = forward(xb, p)
            dz2 = pr.copy(); dz2[np.arange(len(b)), yb] -= 1; dz2 /= len(b)
            g = {"W2": h.T @ dz2, "b2": dz2.sum(0)}
            dh = dz2 @ p["W2"].T; dh[h <= 0] = 0
            g["W1"], g["b1"] = xb.T @ dh, dh.sum(0)
            t += 1
            for k in p:  # Adam update
                m[k] = 0.9 * m[k] + 0.1 * g[k]
                v[k] = 0.999 * v[k] + 0.001 * g[k] ** 2
                p[k] -= LR * (m[k] / (1 - 0.9 ** t)) / (np.sqrt(v[k] / (1 - 0.999 ** t)) + 1e-8)
        if ep % 20 == 0 or ep == 1:
            tra = (forward(Xtr, p)[1].argmax(1) == ytr).mean()
```

```
            tea = (forward(Xte, p)[1].argmax(1) == yte).mean()
            print(f"epoch {ep:3d}  train acc {tra:.4f}  test acc {tea:.4f}")

    train_acc = float((forward(Xtr, p)[1].argmax(1) == ytr).mean())
    test_acc = float((forward(Xte, p)[1].argmax(1) == yte).mean())
    params = int(sum(v_.size for v_ in p.values()))

    # one held-out test example per digit, served by GET /samples for easy verification
    sx, sy = [], []
    for digit in range(10):
        j = np.where(yte == digit)[0][0]
        sx.append(d.data[te[j]]); sy.append(digit)

    meta = {"name": "digits-mlp", "dataset": "UCI handwritten digits (sklearn load_digits), 8x8 grayscale",
            "architecture": "Input 64 -> Dense 64 + ReLU -> Dense 10 + Softmax", "parameters": params,
            "input": "64 pixel values in the range 0-16 (flattened 8x8 image)", "classes": list(range(10)),
            "train_samples": int(len(tr)), "test_samples": int(len(te)),
            "train_accuracy": round(train_acc, 4), "test_accuracy": round(test_acc, 4),
            "epochs": EPOCHS, "seed": SEED, "trained_on": str(date.today()), "numpy_version": np.__version__}
    os.makedirs(os.path.dirname(OUT), exist_ok=True)
    np.savez(OUT, W1=p["W1"].astype(np.float32), b1=p["b1"].astype(np.float32),
             W2=p["W2"].astype(np.float32), b2=p["b2"].astype(np.float32),
             samples_x=np.array(sx, dtype=np.float32), samples_y=np.array(sy, dtype=np.int64),
             meta=np.array(json.dumps(meta)))
    print(json.dumps(meta, indent=2))
    print("saved", OUT, f"({os.path.getsize(OUT) / 1024:.1f} KB)")


if __name__ == "__main__":
    main()
```

5.3 test_api.py - API tests

```
#!/usr/bin/env python3
"""
test_api.py - black-box tests for the running API (standard library only).

Usage:  python test_api.py --url http://localhost:5000 --out api_test_results.json
Works against a local server, gunicorn, or the Docker container (port 5000 published).
"""
import argparse
import json
import statistics
import sys
import time
import urllib.error
import urllib.request


def call(base, path, method="GET", body=None, raw=None, headers=None):
    data = raw if raw is not None else (json.dumps(body).encode() if body is not None else None)
    h = {"Content-Type": "application/json"} if data is not None else {}
    h.update(headers or {})
    req = urllib.request.Request(base + path, data=data, method=method, headers=h)
    t0 = time.perf_counter()
    try:
        with urllib.request.urlopen(req, timeout=10) as r:
            status, text = r.status, r.read().decode()
    except urllib.error.HTTPError as e:
        status, text = e.code, e.read().decode()
    ms = (time.perf_counter() - t0) * 1000
    try:
        parsed = json.loads(text)
    except ValueError:
        parsed = text
```

```
    return status, parsed, ms


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--url", default="http://localhost:5000")
    ap.add_argument("--out", default="api_test_results.json")
    a = ap.parse_args()
    base, checks = a.url.rstrip("/"), []

    def check(name, ok, detail):
        checks.append({"test": name, "passed": bool(ok), "detail": detail})
        print(f"[{'PASS' if ok else 'FAIL'}] {name}: {detail}")

    s, b, _ = call(base, "/health")
    check("GET /health returns 200 and status ok", s == 200 and b.get("status") == "ok", f"HTTP {s} {json.dumps(b)}")
    s, b, _ = call(base, "/")
    check("GET / lists the endpoints", s == 200 and "endpoints" in b, f"HTTP {s}, endpoints={list(b.get('endpoints', {}))}")
    s, info, _ = call(base, "/model-info")
    check("GET /model-info returns model metadata", s == 200 and info.get("parameters") == 4810,
          f"HTTP {s}, parameters={info.get('parameters')}, test_accuracy={info.get('test_accuracy')}")
    s, samples, _ = call(base, "/samples")
    check("GET /samples returns 10 labelled examples", s == 200 and len(samples) == 10, f"HTTP {s}, {len(samples)} samples")

    correct, rows = 0, []
    for smp in samples:
        s, r, _ = call(base, "/predict", "POST", {"pixels": smp["pixels"]})
        ok = s == 200 and r.get("digit") == smp["label"]
        correct += ok
        rows.append({"true": smp["label"], "predicted": r.get("digit"), "confidence": r.get("confidence")})
    check("POST /predict classifies the 10 samples", correct == 10, f"{correct}/10 correct")
    for row in rows: print(f"        true {row['true']} -> predicted {row['predicted']} (confidence {row['confidence']})")

    s, r, _ = call(base, "/predict", "POST", {"instances": [x["pixels"] for x in samples]})
    batch_ok = s == 200 and r.get("count") == 10 and [p["digit"] for p in r["predictions"]] == [x["label"] for x in samples]
    check("POST /predict batch of 10 instances", batch_ok, f"HTTP {s}, count={r.get('count')}, inference_ms={r.get('inference_ms')}")

    s, r, _ = call(base, "/predict", "POST", {"pixels": [1, 2, 3]})
    check("Wrong input length is rejected with 400", s == 400, f"HTTP {s} {json.dumps(r)}")
    s, r, _ = call(base, "/predict", "POST", {"pixels": ["a"] * 64})
    check("Non-numeric pixels are rejected with 400", s == 400, f"HTTP {s} {json.dumps(r)}")
    s, r, _ = call(base, "/predict", "POST", {"pixels": [99] * 64})
    check("Out-of-range pixels are rejected with 400", s == 400, f"HTTP {s} {json.dumps(r)}")
    s, r, _ = call(base, "/predict", "POST", raw=b"not json")
    check("Invalid JSON is rejected with 400", s == 400, f"HTTP {s} {json.dumps(r)}")
    s, r, _ = call(base, "/nope")
    check("Unknown route returns JSON 404", s == 404 and isinstance(r, dict), f"HTTP {s} {json.dumps(r)}")
    s, r, _ = call(base, "/predict")
    check("GET on /predict returns JSON 405", s == 405 and isinstance(r, dict), f"HTTP {s} {json.dumps(r)}")

    lat = [call(base, "/predict", "POST", {"pixels": samples[i % 10]["pixels"]})[2] for i in range(50)]
    summary = {"requests": 50, "mean_ms": round(statistics.mean(lat), 2), "median_ms": round(statistics.median(lat), 2),
               "max_ms": round(max(lat), 2)}
    check("Latency of 50 sequential /predict calls measured", True, json.dumps(summary))

    passed = sum(c["passed"] for c in checks)
    print(f"\n{passed}/{len(checks)} checks passed")
    json.dump({"base_url": base, "passed": passed, "total": len(checks), "checks": checks, "predictions": rows,
               "latency": summary}, open(a.out, "w"), indent=2)
    sys.exit(0 if passed == len(checks) else 1)


if __name__ == "__main__":
    main()
```

5.4 Dockerfile - optimized multi-stage image

```
# ---------- Stage 1: build (install dependencies into an isolated prefix) ----------
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ---------- Stage 2: runtime (only what is needed to serve the API) ----------
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PORT=5000

# run as an unprivileged user instead of root
RUN useradd --create-home --uid 10001 --shell /usr/sbin/nologin appuser

WORKDIR /app
COPY --from=builder /install /usr/local
COPY app.py .
COPY model/ model/

USER appuser
EXPOSE 5000

HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
  CMD ["python", "-c", "import urllib.request;
↳    urllib.request.urlopen('http://127.0.0.1:5000/health', timeout=2)"]

# production WSGI server (not the Flask development server)
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "--access-logfile", "-", "app:app"]
```

5.5 Dockerfile.baseline - un-optimized reference

```
# Baseline (un-optimized) image used only to measure the effect of the optimizations.
FROM python:3.12
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```

5.6 .dockerignore

```
# keep the build context (and therefore the image) small
.git
.gitignore
__pycache__/
*.pyc
venv/
.venv/
*.md
*.pdf
*.json
*.txt
!requirements.txt
screenshots/
train_model.py
test_api.py
task10_docker.py
requirements-train.txt
Dockerfile*
.dockerignore
```

5.7 requirements.txt

```
blinker==1.9.0
click==8.5.0
Flask==3.1.3
gunicorn==26.2.0
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.4
numpy==2.5.3
Werkzeug==3.1.9
```

5.8 task10_docker.py - Docker workflow runner

```
#!/usr/bin/env python3
"""
task10_docker.py - runs the complete Docker workflow for Task 10 (Containerizing a Flask
Deep Learning API) and records the REAL output of every command.

Steps: build the baseline and the optimized image, compare sizes, run the container locally,
verify the API (curl + test_api.py), inspect the container, compare with the baseline, clean up.

Usage:
    1. Start Docker Desktop and wait for "Engine running".
    2. Run from the project folder (the one containing Dockerfile and app.py):
           python3 task10_docker.py
Outputs:
    docker_command_log.txt   plain-text log of every command, output and exit code
    docker_results.json      structured results (used to build the report)
    api_test_results.json    results of test_api.py against the running container
    image_size_comparison.txt  baseline vs optimized image size
Host ports used: 8000 (optimized container) and 8001 (baseline container); the container
itself listens on 5000. Port 5000 is avoided on the host because macOS uses it for AirPlay.
"""
import json
import os
import subprocess
import sys
from datetime import datetime

HERE = os.path.dirname(os.path.abspath(__file__))
os.chdir(HERE)

SAMPLE = '{"pixels": [0.0, 1.0, 11.0, 15.0, 16.0, 9.0, 0.0, 0.0, 0.0, 3.0, 16.0, 10.0, 10.0, 16.0,
    1.0, 0.0, 0.0, 0.0, 2.0, 1.0, 14.0, 11.0, 0.0, 0.0, 0.0, 0.0, 0.0, 14.0, 16.0, 7.0, 0.0, 0.0,
    0.0, 0.0, 0.0, 13.0, 16.0, 16.0, 5.0, 0.0, 0.0, 0.0, 0.0, 0.0, 2.0, 16.0, 8.0, 0.0, 0.0, 0.0,
    6.0, 8.0, 13.0, 15.0, 5.0, 0.0, 0.0, 0.0, 15.0, 16.0, 12.0, 5.0, 0.0, 0.0]}'
WAIT = ("for i in $(seq 1 40); do s=$(docker inspect --format '{{.State.Health.Status}}' %s); "
        "echo \"check $i: $s\"; [ \"$s\" = healthy ] && break; sleep 2; done; [ \"$s\" = healthy ]")
F = "table {{.Names}}\\t{{.Image}}\\t{{.Status}}\\t{{.Ports}}"

A = "1. Verify Docker"
B = "2. Build the images and compare their size"
C = "3. Run the container locally"
D = "4. Verify API functionality"
E = "5. Inspect the container and check the optimizations"
G = "6. Compare with the baseline container"
H = "7. Cleanup"

STEPS = [
    (A, "Docker version", "docker --version && docker version --format 'Client: {{.Client.Version}}
     | Server: {{.Server.Version}}'"),
    (A, "Engine information", "docker info --format 'Containers: {{.Containers}} | Images:
     {{.Images}} | Storage driver: {{.Driver}} | OS: {{.OperatingSystem}}'"),
    (B, "Build the baseline image (un-optimized Dockerfile)", "docker build -f Dockerfile.baseline
     -t flask-dl-api:baseline ."),
    (B, "Build the optimized image (multi-stage Dockerfile)", "docker build -t flask-dl-api:1.0 ."),
    (B, "List the images", "docker images flask-dl-api --format 'table
     {{.Repository}}\\t{{.Tag}}\\t{{.ID}}\\t{{.Size}}'"),
    (B, "Image size of the baseline in bytes", "docker image inspect flask-dl-api:baseline --format
     '{{.Size}}'"),
    (B, "Image size of the optimized image in bytes", "docker image inspect flask-dl-api:1.0
     --format '{{.Size}}'"),
    (B, "Layer history of the optimized image", "docker history flask-dl-api:1.0"),
    (C, "Start the container (host port 8000 -> container port 5000)", "docker run -d --name flask-
     dl-api -p 8000:5000 flask-dl-api:1.0"),
    (C, "Wait until the HEALTHCHECK reports healthy", WAIT % "flask-dl-api"),
    (C, "List the running container", "docker ps --filter name=flask-dl-api --format '" + F + "'"),
```

```
    (D, "GET /health", "curl -s http://localhost:8000/health"),
    (D, "GET /model-info", "curl -s http://localhost:8000/model-info"),
    (D, "POST /predict with a sample image of the digit 3", "curl -s -X POST -H 'Content-Type: application/json' -d '" + SAMPLE + "' http://localhost:8000/predict"),
    (D, "POST /predict with invalid input (expect HTTP 400)", "curl -s -w '\\nHTTP status: %{http_code}\\n' -X POST -H 'Content-Type: application/json' -d '{\"pixels\":[1,2,3]}' http://localhost:8000/predict"),
    (D, "Full API test suite (test_api.py)", "python3 test_api.py --url http://localhost:8000 --out api_test_results.json"),
    (E, "User inside the container (should not be root)", "docker exec flask-dl-api id"),
    (E, "Container configuration and health", "docker inspect flask-dl-api --format 'User={{.Config.User}} | Health={{.State.Health.Status}} | Image={{.Config.Image}}'"),
    (E, "Training-only dependency is absent from the image", "docker exec flask-dl-api python -c \"import importlib.util as u; print('scikit-learn installed:', u.find_spec('sklearn') is not None); print('numpy installed:', u.find_spec('numpy') is not None)\""),
    (E, "Files inside the image", "docker exec flask-dl-api ls -la /app /app/model"),
    (E, "Container logs (gunicorn access log)", "docker logs --tail 12 flask-dl-api"),
    (E, "Resource usage", "docker stats --no-stream flask-dl-api"),
    (E, "Restart the container and check that the API recovers", "docker restart flask-dl-api && " + WAIT % "flask-dl-api" + " && curl -s http://localhost:8000/health"),
    (G, "Start the baseline container (host port 8001)", "docker run -d --name flask-dl-api-baseline -p 8001:5000 flask-dl-api:baseline && sleep 4 && docker ps --filter name=flask-dl-api-baseline --format '" + F + "'"),
    (G, "Baseline API responds", "curl -s http://localhost:8001/health"),
    (G, "User inside the baseline container", "docker exec flask-dl-api-baseline id"),
    (G, "Resource usage of both containers", "docker stats --no-stream flask-dl-api flask-dl-api-baseline"),
    (H, "Remove the containers (images are kept)", "docker rm -f flask-dl-api flask-dl-api-baseline && docker ps -a --format 'table {{.Names}}\\t{{.Status}}'"),
]


def run_cmd(cmd, timeout=1500):
    t0 = datetime.now()
    try:
        p = subprocess.run(["bash", "-c", cmd], capture_output=True, text=True, timeout=timeout)
        out = (p.stdout or "") + (("\n[stderr]\n" + p.stderr) if p.stderr.strip() else "")
        rc = p.returncode
    except subprocess.TimeoutExpired:
        out, rc = "Command timed out.", 124
    return {"command": cmd, "output": out.rstrip(), "returncode": rc, "time": t0.strftime("%Y-%m-%d %H:%M:%S")}


def main():
    if run_cmd("docker info", 60)["returncode"] != 0:
        sys.exit("Docker is not running. Start Docker Desktop, wait for 'Engine running', then run again.")
    for f in ("Dockerfile", "Dockerfile.baseline", "app.py", "requirements.txt", "model/model.npz", "test_api.py"):
        if not os.path.exists(f):
            sys.exit(f"Missing file: {f}. Run this script from the project folder.")
    results = []
    for i, (section, title, cmd) in enumerate(STEPS, 1):
        r = run_cmd(cmd)
        r.update({"section": section, "title": title})
        results.append(r)
        print(f"\n### [{i}/{len(STEPS)}] {title}\n$ {cmd}\n{r['output']}\n(exit code {r['returncode']})")

    # image size comparison
    def size(title):
        r = next(x for x in results if x["title"] == title)
        digits = "".join(ch for ch in r["output"].split("\n")[0] if ch.isdigit())
        return int(digits) if digits else None
    sb, so = size("Image size of the baseline in bytes"), size("Image size of the optimized image in bytes")
    if sb and so:
        txt = (f"baseline  (python:3.12, single stage, root)      : {sb:>13,} bytes = {sb / 1e6:8.1f} MB\n"
               f"optimized (python:3.12-slim, multi-stage, user)   : {so:>13,} bytes = {so / 1e6:8.1f} MB\n"
               f"reduction                                          : {(sb - so) / sb * 100:6.1f} % ({(sb - so) / 1e6:.1f} MB smaller)\n")
        open("image_size_comparison.txt", "w").write(txt)
        print("\n" + txt)

    json.dump(results, open("docker_results.json", "w"), indent=2)
    with open("docker_command_log.txt", "w") as f:
        f.write(f"Task 10 Docker command log - generated {datetime.now():%Y-%m-%d %H:%M:%S}\n" + "="
```

```
    ↳     * 78 + "\n")
115         for i, r in enumerate(results, 1):
116             f.write(f"\n[{i}] {r['section']} - {r['title']}  ({r['time']})\n$
    ↳    {r['command']}\n{r['output']}\n"
117                     f"(exit code {r['returncode']})\n" + "-" * 78 + "\n")
118     ok = sum(1 for r in results if r["returncode"] == 0)
119     print(f"\nDone: {ok}/{len(results)} commands succeeded. Log: docker_command_log.txt, results:
    ↳    docker_results.json")
120
121
122 if __name__ == "__main__":
123     main()
```

6. Command Execution Log

All 28 commands below were run by `task10_docker.py`. The command, output and exit code are copied from `docker_results.json`; the observation under each step explains what the output shows. The long build outputs are shortened (marked with '...'); the complete text is in `docker_command_log.txt`.

6.1 Verify Docker (Steps 1-2)

Concept: The docker CLI sends requests to the Docker daemon, which builds images and runs containers. Docker Desktop provides the daemon on macOS inside a lightweight Linux VM.

Step 1: Docker version (exit code 0)

```
$ docker --version && docker version --format 'Client: {{.Client.Version}} | Server: {{.Server.Version}}'
```

```
Docker version 29.8.2, build 7fc2dff
Client: 29.8.2 | Server: 29.8.2
```

Observation: Client and server both report version 29.8.2, so the CLI is talking to the Docker Desktop engine correctly.

Step 2: Engine information (exit code 0)

```
$ docker info --format 'Containers: {{.Containers}} | Images: {{.Images}} | Storage driver: {{.Driver}} | OS:
  {{.OperatingSystem}}'
```

```
Containers: 0 | Images: 20 | Storage driver: overlayfs | OS: Docker Desktop
```

Observation: Before the build the engine had 0 running containers and 20 images. The storage driver is overlayfs and the operating system is reported as Docker Desktop (the engine runs inside Docker Desktop's Linux VM).

6.2 Build the Images and Compare Their Size (Steps 3-8)

Concept: A Dockerfile lists the instructions that build an image; every instruction creates a cached layer. A multi-stage build installs dependencies in a builder stage and copies only the result into a small final stage, so build tools and caches do not end up in the image that is deployed.

Step 3: Build the baseline image (un-optimized Dockerfile) (exit code 0)

```
$ docker build -f Dockerfile.baseline -t flask-dl-api:baseline .
```

```
[stderr]
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile.baseline
#1 transferring dockerfile: 246B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/python:3.12
... [28 lines omitted - full output in docker_command_log.txt]

#10 exporting to image
#10 exporting layers done
#10 exporting manifest sha256:8f147c2c672e67b0c51df0c64e6f9d496c4cccecb7e7deb67bd2950cc06c42c1 done
#10 exporting config sha256:d93fb7669780331d1b39ad89a12da3ba2a1f69d5768ee19f8602472911b43daf done
#10 exporting attestation manifest sha256:397dfe875565822f4c7c0c3237bb918591f4cd8c9c0a5cbee0f2b127bb4f1cad done
#10 exporting manifest list sha256:af5ebbd69f5f404cce3962489f2b74b7e002de710381efcd9d13cefbb8582f4c done
#10 naming to docker.io/library/flask-dl-api:baseline done
#10 unpacking to docker.io/library/flask-dl-api:baseline done
#10 DONE 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/c7rgum36qxd55a65a7gspssbs
```

Observation: The un-optimized reference image was built from Dockerfile.baseline (FROM python:3.12, COPY . ., pip install, python app.py) and tagged flask-dl-api:baseline. The COPY and pip steps show CACHED because this image had been built before, so the build duration is not meaningful. BuildKit writes its progress to stderr; the exit code is 0.

Step 4: Build the optimized image (multi-stage Dockerfile) (exit code 0)

```
$ docker build -t flask-dl-api:1.0 .
```

```
[stderr]
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 1.02kB done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/python:3.12-slim
... [76 lines omitted - full output in docker_command_log.txt]
#14 exporting layers
#14 exporting layers 2.3s done
#14 exporting manifest sha256:f9952b2ddf954f47ecee6db10868a99e576ffae03389fd46e0b161e502c5a25b done
#14 exporting config sha256:79cdd41ec6b7f85ee077e2c2c1897af577931be042c3985877013ca843dcf310 done
#14 exporting attestation manifest sha256:014210c574f035d5604e35464af8a6403d95199f49fdf0e58faab3d36a28d48d done
#14 exporting manifest list sha256:2f46c8e2450aa0e7e55e2bdf45dbe8c4b46397acc129c2a85acac7b6f86ca3ff done
#14 naming to docker.io/library/flask-dl-api:1.0 done
#14 unpacking to docker.io/library/flask-dl-api:1.0
#14 unpacking to docker.io/library/flask-dl-api:1.0 0.6s done
#14 DONE 2.9s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/f22x91f0fhwqp8yjh1bbnqxt6
```

Observation: The optimized image was built in two stages. In the builder stage the dependencies were installed into /install (pip install, 10.9 s). In the runtime stage the user appuser was created and /install, app.py and model/ were copied in. The image was exported and named flask-dl-api:1.0 (export 2.9 s). Lines marked CACHED reuse layers from an earlier build.

Step 5: List the images (exit code 0)

```
$ docker images flask-dl-api --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}\t{{.Size}}'
```

```
REPOSITORY     TAG        IMAGE ID       SIZE
flask-dl-api   1.0        2f46c8e2450a   303MB
flask-dl-api   baseline   af5ebbd69f5f   1.75GB
```

Observation: Two images exist: flask-dl-api:1.0 (optimized, 303 MB) and flask-dl-api:baseline (1.75 GB).

Step 6: Image size of the baseline in bytes (exit code 0)

```
$ docker image inspect flask-dl-api:baseline --format '{{.Size}}'
```

```
1752970368
```

Observation: The baseline image occupies 1,752,970,368 bytes (about 1,753.0 MB).

Step 7: Image size of the optimized image in bytes (exit code 0)

```
$ docker image inspect flask-dl-api:1.0 --format '{{.Size}}'
```

```
303293360
```

Observation: The optimized image occupies 303,293,360 bytes (about 303.3 MB), which is 82.7 % (1,449.7 MB) smaller than the baseline.

Step 8: Layer history of the optimized image (exit code 0)

```
$ docker history flask-dl-api:1.0
```

```
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
2f46c8e2450a   3 seconds ago    CMD ["gunicorn" "--bind" "0.0.0.0:5000" "--w…   0B        buildkit.dockerfile.v0
<missing>      3 seconds ago    HEALTHCHECK {Test:[CMD python -c import urll…   0B        buildkit.dockerfile.v0
<missing>      3 seconds ago    EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      3 seconds ago    USER appuser                                    0B        buildkit.dockerfile.v0
<missing>      3 seconds ago    COPY model/ model/ # buildkit                   41kB      buildkit.dockerfile.v0
<missing>      3 seconds ago    COPY app.py . # buildkit                        16.4kB    buildkit.dockerfile.v0
<missing>      3 seconds ago    COPY /install /usr/local # buildkit             78.3MB    buildkit.dockerfile.v0
<missing>      15 seconds ago   WORKDIR /app                                    8.19kB    buildkit.dockerfile.v0
<missing>      15 seconds ago   RUN /bin/sh -c useradd --create-home --uid 1…   69.6kB    buildkit.dockerfile.v0
<missing>      15 seconds ago   ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFER…   0B        buildkit.dockerfile.v0
<missing>      3 days ago       CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      3 days ago       RUN /bin/sh -c set -eux;  for src in idle3 p…   16.4kB    buildkit.dockerfile.v0
<missing>      3 days ago       RUN /bin/sh -c set -eux;   savedAptMark="$(a…   44.6MB    buildkit.dockerfile.v0
<missing>      3 days ago       ENV PYTHON_SHA256=c2c4321961fab0fb999d66e0ce…   0B        buildkit.dockerfile.v0
<missing>      3 days ago       ENV PYTHON_VERSION=3.12.15                      0B        buildkit.dockerfile.v0
<missing>      3 days ago       ENV GPG_KEY=7169605F62C751356D054A26A821E680…   0B        buildkit.dockerfile.v0
<missing>      3 days ago       RUN /bin/sh -c set -eux;  apt-get update;  a…   4.99MB    buildkit.dockerfile.v0
<missing>      3 days ago       ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      3 days ago       ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      4 days ago       # debian.sh --arch 'arm64' out/ 'trixie' '@1…   110MB     debuerreotype 0.17
```

Observation: The history shows the base layers of python:3.12-slim (110 MB Debian arm64 layer, 44.6 MB Python layer, 4.99 MB apt layer, Python 3.12.15) followed by the application layers: useradd 69.6 kB, the installed packages copied from the builder stage 78.3 MB, app.py 16.4 kB and the model 41 kB. USER, EXPOSE, HEALTHCHECK and CMD only set metadata (0 B).

6.3 Run the Container Locally (Steps 9-11)

Concept: A container is a running instance of an image. The option -p host:container publishes the container's port on the host, and a HEALTHCHECK lets Docker report whether the application inside responds.

Step 9: Start the container (host port 8000 -> container port 5000) (exit code 0)

```
$ docker run -d --name flask-dl-api -p 8000:5000 flask-dl-api:1.0
```

```
a82e4686a00ae31bd7b66ac1c9ade77881f856301854cbdf2e6041c2fef50ca2
```

Observation: The container was started in the background. Host port 8000 is mapped to container port 5000 (port 5000 itself is not used on the Mac because macOS uses it for AirPlay Receiver). The command prints the full container ID.

Step 10: Wait until the HEALTHCHECK reports healthy (exit code 0)

```
$ for i in $(seq 1 40); do s=$(docker inspect --format '{{.State.Health.Status}}' flask-dl-api); echo "check $i:
   $s"; [ "$s" = healthy ] && break; sleep 2; done; [ "$s" = healthy ]
```

```
check 1: starting
check 2: starting
check 3: starting
check 4: healthy
```

Observation: The HEALTHCHECK status moved from starting (checks 1 to 3) to healthy at check 4, about 6 seconds after the start.

Step 11: List the running container (exit code 0)

```
$ docker ps --filter name=flask-dl-api --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
```

```
NAMES          IMAGE              STATUS                    PORTS
flask-dl-api   flask-dl-api:1.0   Up 6 seconds (healthy)    0.0.0.0:8000->5000/tcp, [::]:8000->5000/tcp
```

Observation: docker ps lists the container as 'Up 6 seconds (healthy)' with the port mapping 0.0.0.0:8000->5000/tcp for IPv4 and [::]:8000->5000/tcp for IPv6.

6.4 Verify API Functionality (Steps 12-16)

Concept: The API is tested from outside the container through the published port: the GET routes, predictions for known sample digits, and the answers to invalid requests.

Step 12: GET /health (exit code 0)

```
$ curl -s http://localhost:8000/health
```

```
{"status":"ok","uptime_seconds":5.5}
```

Observation: GET /health returns {"status":"ok"} with the uptime of the server process (5.5 s).

Step 13: GET /model-info (exit code 0)

```
$ curl -s http://localhost:8000/model-info
```

```
{"architecture":"Input 64 -> Dense 64 + ReLU -> Dense 10 +
  Softmax","classes":[0,1,2,3,4,5,6,7,8,9],"dataset":"UCI handwritten digits (sklearn load_digits), 8x8
  grayscale","epochs":80,"input":"64 pixel values in the range 0-16 (flattened 8x8 image)","name":"digits-mlp","
  numpy_version":"2.4.4","parameters":4810,"seed":42,"test_accuracy":0.9722,"test_samples":360,"train_accuracy":
  1.0,"train_samples":1437,"trained_on":"2026-10-09"}
```

Observation: GET /model-info returns the model metadata: digits-mlp, architecture 64 -> 64 (ReLU) -> 10 (softmax), 4,810 parameters, 80 epochs, 1,437 training and 360 test samples, test accuracy 0.9722.

Step 14: POST /predict with a sample image of the digit 3 (exit code 0)

```
$ curl -s -X POST -H 'Content-Type: application/json' -d '{"pixels": [0.0, 1.0, 11.0, 15.0, 16.0, 9.0, 0.0, 0.0,
   0.0, 3.0, 16.0, 10.0, 10.0, 16.0, 1.0, 0.0, 0.0, 0.0, 2.0, 1.0, 14.0, 11.0, 0.0, 0.0, 0.0, 0.0, 14.0,
  16.0, 7.0, 0.0, 0.0, 0.0, 0.0, 0.0, 13.0, 16.0, 16.0, 5.0, 0.0, 0.0, 0.0, 0.0, 2.0, 16.0, 8.0, 0.0, 0.0,
  0.0, 6.0, 8.0, 13.0, 15.0, 5.0, 0.0, 0.0, 0.0, 15.0, 16.0, 12.0, 5.0, 0.0, 0.0]}'
  http://localhost:8000/predict
```

```
{"confidence":1.0,"count":1,"digit":3,"inference_ms":0.912,"predictions":[{"confidence":1.0,"digit":3,"probabili
  ties":[0.0,0.0,0.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0]}],"probabilities":[0.0,0.0,0.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0]}
```

Observation: A sample image of the digit 3 is classified as 3 with confidence 1.0; the model needed 0.912 ms for the inference. The response contains the single-image fields (digit, confidence, probabilities) and the predictions list.

Step 15: POST /predict with invalid input (expect HTTP 400) (exit code 0)

```
$ curl -s -w '\nHTTP status: %{http_code}\n' -X POST -H 'Content-Type: application/json' -d '{"pixels":[1,2,3]}'
   http://localhost:8000/predict
```

```
{"error":"Each image needs exactly 64 pixel values (flattened 8x8)."}

HTTP status: 400
```

Observation: An image with only 3 pixel values is rejected: the API answers with a JSON error message and HTTP status 400.

Step 16: Full API test suite (test_api.py) (exit code 0)

```
$ python3 test_api.py --url http://localhost:8000 --out api_test_results.json
```

```
[PASS] GET /health returns 200 and status ok: HTTP 200 {"status": "ok", "uptime_seconds": 6.4}
[PASS] GET / lists the endpoints: HTTP 200, endpoints=['GET /health', 'GET /model-info', 'GET /samples', 'POST
  /predict']
[PASS] GET /model-info returns model metadata: HTTP 200, parameters=4810, test_accuracy=0.9722
[PASS] GET /samples returns 10 labelled examples: HTTP 200, 10 samples
[PASS] POST /predict classifies the 10 samples: 10/10 correct
        true 0 -> predicted 0 (confidence 1.0)
        true 1 -> predicted 1 (confidence 0.9989)
        true 2 -> predicted 2 (confidence 1.0)
        true 3 -> predicted 3 (confidence 1.0)
        true 4 -> predicted 4 (confidence 1.0)
        true 5 -> predicted 5 (confidence 1.0)
        true 6 -> predicted 6 (confidence 0.9972)
        true 7 -> predicted 7 (confidence 1.0)
        true 8 -> predicted 8 (confidence 0.9951)
        true 9 -> predicted 9 (confidence 0.9837)
[PASS] POST /predict batch of 10 instances: HTTP 200, count=10, inference_ms=1.131
[PASS] Wrong input length is rejected with 400: HTTP 400 {"error": "Each image needs exactly 64 pixel values
  (flattened 8x8)."}
[PASS] Non-numeric pixels are rejected with 400: HTTP 400 {"error": "All pixel values must be numbers."}
[PASS] Out-of-range pixels are rejected with 400: HTTP 400 {"error": "Pixel values must be finite numbers
  between 0 and 16."}
[PASS] Invalid JSON is rejected with 400: HTTP 400 {"error": "Request body must be a JSON object with \"pixels\"
   or \"instances\"."}
[PASS] Unknown route returns JSON 404: HTTP 404 {"error": "Route not found."}
[PASS] GET on /predict returns JSON 405: HTTP 405 {"error": "Method not allowed for this route."}
[PASS] Latency of 50 sequential /predict calls measured: {"requests": 50, "mean_ms": 1.36, "median_ms": 1.3,
  "max_ms": 2.38}
13/13 checks passed
```

Observation: All 13 checks passed: the four GET routes, 10 of 10 sample digits classified correctly, a batch of 10 instances, four kinds of invalid input answered with 400, JSON 404 and 405 errors, and the latency of 50 sequential requests (mean 1.36 ms, median 1.30 ms, maximum 2.38 ms).

6.5 Inspect the Container and Check the Optimizations (Steps 17-23)

Concept: These steps check the optimization practices on the running container: non-root user, health status, absence of training-only dependencies, image contents, logs, resource use and recovery after a restart.

Step 17: User inside the container (should not be root) (exit code 0)

```
$ docker exec flask-dl-api id
```

```
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
```

Observation: The process inside the container runs as uid 10001 (appuser), not as root.

Step 18: Container configuration and health (exit code 0)

```
$ docker inspect flask-dl-api --format 'User={{.Config.User}} | Health={{.State.Health.Status}} |
  Image={{.Config.Image}}'
```

```
User=appuser | Health=healthy | Image=flask-dl-api:1.0
```

Observation: docker inspect confirms User=appuser, Health=healthy and Image=flask-dl-api:1.0.

Step 19: Training-only dependency is absent from the image (exit code 0)

```
$ docker exec flask-dl-api python -c "import importlib.util as u; print('scikit-learn installed:',
  u.find_spec('sklearn') is not None); print('numpy installed:', u.find_spec('numpy') is not None)"
```

```
scikit-learn installed: False
numpy installed: True
```

Observation: scikit-learn, which is only needed for training, is not installed in the image, while NumPy is. This keeps the image small.

Step 20: Files inside the image (exit code 0)

```
$ docker exec flask-dl-api ls -la /app /app/model
```

```
/app:
total 20
drwxr-xr-x 1 root root 4096 Oct  9 20:35 .
drwxr-xr-x 1 root root 4096 Oct  9 20:35 ..
-rw-r--r-- 1 root root 4274 Oct  9 02:42 app.py
drwxr-xr-x 2 root root 4096 Oct  9 15:19 model

/app/model:
total 36
drwxr-xr-x 2 root root  4096 Oct  9 15:19 .
drwxr-xr-x 1 root root  4096 Oct  9 20:35 ..
-rw-r--r-- 1 root root 25458 Oct  9 02:44 model.npz
```

Observation: The image contains only app.py (4,274 bytes) and model/model.npz (25,458 bytes) in /app. Training scripts, tests, logs and documents are excluded by .dockerignore. The files belong to root and are read-only for appuser.

Step 21: Container logs (gunicorn access log) (exit code 0)

```
$ docker logs --tail 12 flask-dl-api
```

```
192.168.65.1 - - [09/Oct/2026:20:35:42 +0000] "POST /predict HTTP/1.1" 200 245 "-" "Python-urllib/3.13"
192.168.65.1 - - [09/Oct/2026:20:35:42 +0000] "POST /predict HTTP/1.1" 200 255 "-" "Python-urllib/3.13"
192.168.65.1 - - [09/Oct/2026:20:35:42 +0000] "POST /predict HTTP/1.1" 200 221 "-" "Python-urllib/3.13"
192.168.65.1 - - [09/Oct/2026:20:35:42 +0000] "POST /predict HTTP/1.1" 200 251 "-" "Python-urllib/3.13"
192.168.65.1 - - [09/Oct/2026:20:35:42 +0000] "POST /predict HTTP/1.1" 200 221 "-" "Python-urllib/3.13"
192.168.65.1 - - [09/Oct/2026:20:35:42 +0000] "POST /predict HTTP/1.1" 200 221 "-" "Python-urllib/3.13"
192.168.65.1 - - [09/Oct/2026:20:35:42 +0000] "POST /predict HTTP/1.1" 200 221 "-" "Python-urllib/3.13"
192.168.65.1 - - [09/Oct/2026:20:35:42 +0000] "POST /predict HTTP/1.1" 200 221 "-" "Python-urllib/3.13"
192.168.65.1 - - [09/Oct/2026:20:35:42 +0000] "POST /predict HTTP/1.1" 200 245 "-" "Python-urllib/3.13"
192.168.65.1 - - [09/Oct/2026:20:35:42 +0000] "POST /predict HTTP/1.1" 200 221 "-" "Python-urllib/3.13"
192.168.65.1 - - [09/Oct/2026:20:35:42 +0000] "POST /predict HTTP/1.1" 200 245 "-" "Python-urllib/3.13"
192.168.65.1 - - [09/Oct/2026:20:35:42 +0000] "POST /predict HTTP/1.1" 200 255 "-" "Python-urllib/3.13"
```

Observation: The gunicorn access log shows the POST /predict requests of the test script (HTTP 200) coming from 192.168.65.1, an address of Docker Desktop's internal network.

Step 22: Resource usage (exit code 0)

```
$ docker stats --no-stream flask-dl-api
```

```
CONTAINER ID   NAME           CPU %     MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I/O   PIDS
a82e4686a00a   flask-dl-api   0.02%     97.65MiB / 7.748GiB   1.23%     70.1kB / 61.1kB   31MB / 0B   22
```

Observation: Right after the test run the container used 0.02 % CPU and 97.65 MiB of 7.748 GiB memory (1.23 %), with 22 processes.

Step 23: Restart the container and check that the API recovers (exit code 0)

```
$ docker restart flask-dl-api && for i in $(seq 1 40); do s=$(docker inspect --format '{{.State.Health.Status}}'
   flask-dl-api); echo "check $i: $s"; [ "$s" = healthy ] && break; sleep 2; done; [ "$s" = healthy ] && curl -s
   http://localhost:8000/health
```

```
flask-dl-api
check 1: starting
check 2: starting
check 3: starting
check 4: healthy
{"status":"ok","uptime_seconds":5.6}
```

Observation: After docker restart the container passed through starting again and became healthy at check 4. /health answered with an uptime of 5.6 s, which shows that the server process was restarted and the API recovers without any manual step.

6.6 Compare with the Baseline Container (Steps 24-27)

Concept: The un-optimized baseline image is started next to the optimized one to compare how both behave and what they use.

Step 24: Start the baseline container (host port 8001) (exit code 0)

```
$ docker run -d --name flask-dl-api-baseline -p 8001:5000 flask-dl-api:baseline && sleep 4 && docker ps --filter
   name=flask-dl-api-baseline --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
```

```
4e01bb4c67f7b3f91aeaf66a77e6597c50203270e28da4a2e488154db2dae398
NAMES                  IMAGE                  STATUS         PORTS
flask-dl-api-baseline  flask-dl-api:baseline  Up 4 seconds   0.0.0.0:8001->5000/tcp, [::]:8001->5000/tcp
```

Observation: The baseline container started on host port 8001. docker ps shows only 'Up 4 seconds' without a health status, because the baseline image has no HEALTHCHECK.

Step 25: Baseline API responds (exit code 0)

```
$ curl -s http://localhost:8001/health
```

```
{"status":"ok","uptime_seconds":3.5}
```

Observation: The baseline API answers on /health as well (uptime 3.5 s), so both images work; they differ in size, user and process setup.

Step 26: User inside the baseline container (exit code 0)

```
$ docker exec flask-dl-api-baseline id
```

```
uid=0(root) gid=0(root) groups=0(root)
```

Observation: The baseline container runs as uid 0 (root).

Step 27: Resource usage of both containers (exit code 0)

```
$ docker stats --no-stream flask-dl-api flask-dl-api-baseline
```

```
CONTAINER ID   NAME                   CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O
  PIDS
a82e4686a00a   flask-dl-api           0.03%     68.77MiB / 7.748GiB   0.87%     1.91kB / 796B   0B / 0B
  22
4e01bb4c67f7   flask-dl-api-baseline  0.03%     86.24MiB / 7.748GiB   1.09%     1.47kB / 709B   68.3MB / 0B
  10
```

Observation: In this single idle measurement the optimized container uses 68.77 MiB and 22 processes (gunicorn with 2 workers), the baseline 86.24 MiB and 10 processes (Flask development server). CPU is 0.03 % for both.

6.7 Cleanup (Step 28)

Step 28: Remove the containers (images are kept) (exit code 0)

```
$ docker rm -f flask-dl-api flask-dl-api-baseline && docker ps -a --format 'table {{.Names}}\t{{.Status}}'
```

```
flask-dl-api
flask-dl-api-baseline
NAMES     STATUS
```

Observation: Both containers were removed; docker ps -a lists only the header row, so no containers remain. The two images are kept.

7. Screenshots and Observations

All screenshots were captured on the author's Mac (Docker Desktop v4.94.0) while the optimized container was running. The terminal screenshots are cropped to the text; the last one shows Docker Desktop's Containers tab with an enlarged detail of its table.

Notes for transparency.

- The runner script removes its containers in its last step. For the screenshots the container was therefore started again by hand with `docker run -d --name flask-dl-api -p 8000:5000 flask-dl-api:1.0`. Its ID (fde40c39bd13) differs from the scripted run in the log (a82e4686a00a).
- The `docker build` in Figure 1 was run again before the manual run (all layers CACHED) and produced the image ID 0bdc08db70a1, while the scripted build in step 4 gave 2f46c8e2450a. Both come from the same Dockerfile and have the same size (303 MB).
- Figure 6 and `api_test_results.json` come from the manual run (server uptime 350.8 s); step 16 of the log is the scripted run. Both runs passed 13 of 13 checks.

7.1 Building the Optimized Image

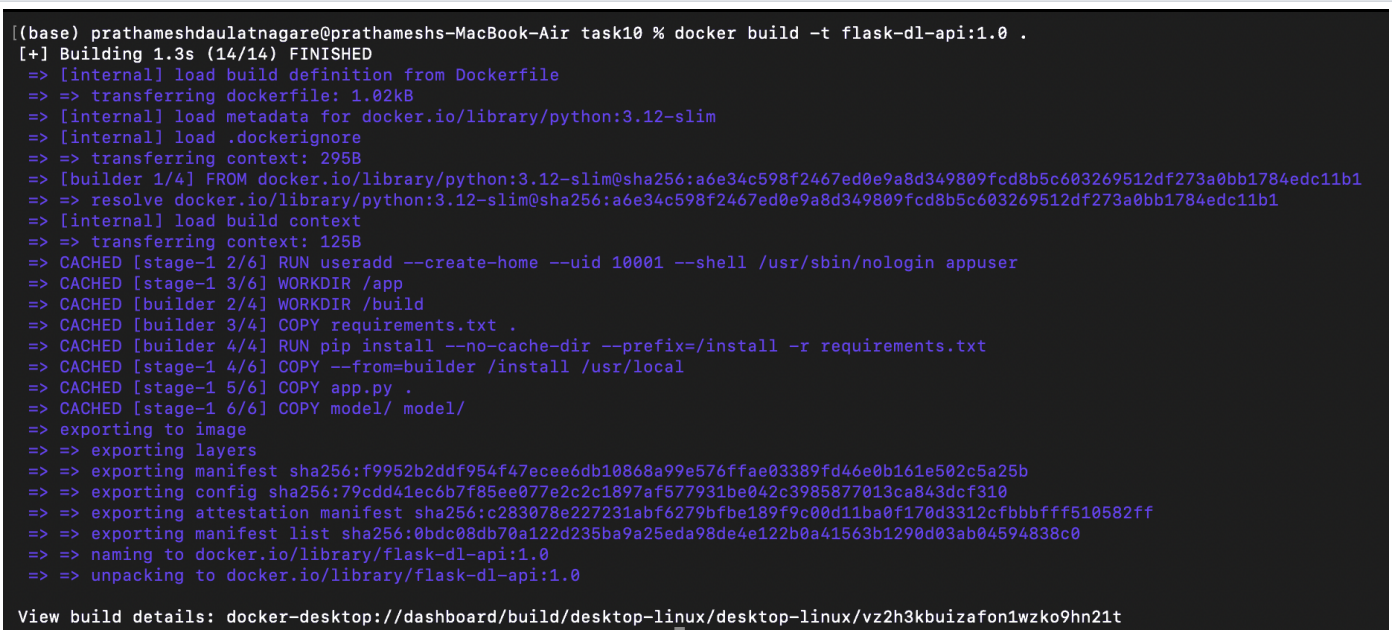


Figure 1: docker build -t flask-dl-api:1.0 . with the two build stages (builder and stage-1) and the export of the image.

What the screenshot shows

- The build finished (14/14 steps) in 1.3 s because every layer was CACHED from the earlier scripted build.
- Builder stage: WORKDIR /build, COPY requirements.txt, RUN pip install --no-cache-dir --prefix=/install. Runtime stage: useradd, WORKDIR /app, COPY --from=builder /install /usr/local, COPY app.py, COPY model/.
- The build context is tiny (295 B and 125 B transferred) because of the .dockerignore file.
- The image is named docker.io/library/flask-dl-api:1.0.

7.2 Image Size: Optimized versus Baseline

```
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task10 % docker images flask-dl-api

IMAGE                  ID             DISK USAGE   CONTENT SIZE   EXTRA
flask-dl-api:1.0       0bdc08db70a1       303MB         65.7MB
flask-dl-api:baseline  af5ebbd69f5f      1.75GB          443MB
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task10 % cat image_size_comparison.txt
baseline  (python:3.12, single stage, root)      : 1,752,970,368 bytes =   1753.0 MB
optimized (python:3.12-slim, multi-stage, user)  :   303,293,360 bytes =    303.3 MB
reduction                                        :   82.7 %  (1449.7 MB smaller)
```

Figure 2: docker images flask-dl-api and the file image_size_comparison.txt written by the runner script.

What the screenshot shows

- flask-dl-api:1.0: ID 0bdc08db70a1, disk usage 303 MB, content size 65.7 MB.
- flask-dl-api:baseline: ID af5ebbd69f5f, disk usage 1.75 GB, content size 443 MB.
- image_size_comparison.txt: 1,752,970,368 bytes (baseline) against 303,293,360 bytes (optimized), a reduction of 82.7 % (1,449.7 MB).

7.3 Starting the Container and Health Status

```
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task10 % docker run -d --name flask-dl-api -p 8000:5000 flask-dl-api:1.0
fde40c39bd13f9afdb19b3101e059ba1a88cd5e27c87f1cd590f65d8980a8988
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task10 % docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED          STATUS                    PORTS                                                NAMES
fde40c39bd13   flask-dl-api:1.0  "gunicorn --bind 0.0.…"  21 seconds ago   Up 20 seconds (healthy)   0.0.0.0:8000->5000/tcp, [::]:8000->5000/tcp   flask-dl-api
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task10 %
```

Figure 3: docker run -d --name flask-dl-api -p 8000:5000 flask-dl-api:1.0, followed by docker ps about 20 seconds later.

What the screenshot shows

- docker run prints the container ID (fde40c39bd13...).
- docker ps: command gunicorn --bind 0.0.0.0:5000, status 'Up 20 seconds (healthy)', ports 0.0.0.0:8000->5000/tcp and [::]:8000->5000/tcp.
- The word (healthy) comes from the HEALTHCHECK in the Dockerfile.

7.4 Health Check and Model Information

```
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task10 % curl http://localhost:8000/health
{"status":"ok","uptime_seconds":89.9}
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task10 % curl http://localhost:8000/model-info
{"architecture":"Input 64 -> Dense 64 + ReLU -> Dense 10 + Softmax","classes":[0,1,2,3,4,5,6,7,8,9],"dataset":"UCI handwritten digits (sklearn load_digits), 8x8 grayscale","epochs":80,"input":"64 pixel values in the range 0-16 (flattened 8x8 image)","name":"digits-mlp","numpy_version":"2.4.4","parameters":4810,"seed":42,"test_accuracy":0.9722,"test_samples":360,"train_accuracy":1.0,"train_samples":1437,"trained_on":"2026-10-09"}
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task10 %
```

Figure 4: curl calls to /health and /model-info on the published port 8000.

What the screenshot shows

- /health returns {"status":"ok"} with the uptime of the server (89.9 s).
- /model-info returns the architecture (Input 64 -> Dense 64 + ReLU -> Dense 10 + Softmax), 4,810 parameters, 80 epochs, 1,437 training and 360 test samples, test accuracy 0.9722 and training accuracy 1.0.

7.5 Prediction and Error Handling

```
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task10 % curl -s -X POST -H 'Content-Type: application/json' -d '{"pixels":[0,1,11,15,16,9,0,0,0,3,16,10,10,16,1,0,0,0,2,1,14,11,0,0,0,0,0,14,16,7,0,0,0,0,0,13,16,16,5,0,0,0,0,0,2,16,8,0,0,0,6,8,13,15,5,0,0,0,15,16,12,5,0,0]}' http://localhost:8000/predict
echo
curl -s -w '\nHTTP status: %{http_code}\n' -X POST -H 'Content-Type: application/json' -d '{"pixels":[1,2,3]}' http://localhost:8000/predict
{"confidence":1.0,"count":1,"digit":3,"inference_ms":1.278,"predictions":[{"confidence":1.0,"digit":3,"probabilities":[0.0,0.0,0.0,0.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0]}],"probabilities":[0.0,0.0,0.0,0.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0]}

{"error":"Each image needs exactly 64 pixel values (flattened 8x8)."}

HTTP status: 400
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task10 %
```

Figure 5: POST /predict with an 8x8 image of the digit 3, and a POST with only three pixel values.

What the screenshot shows

- The valid request returns digit 3 with confidence 1.0 and an inference time of 1.278 ms; the probability vector has its 1.0 at index 3.
- The invalid request returns the JSON error 'Each image needs exactly 64 pixel values (flattened 8x8).' and HTTP status 400.

7.6 API Test Suite

```
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task10 % python3 test_api.py --url http://localhost:8000
[PASS] GET /health returns 200 and status ok: HTTP 200 {"status": "ok", "uptime_seconds": 350.8}
[PASS] GET / lists the endpoints: HTTP 200, endpoints=['GET /health', 'GET /model-info', 'GET /samples', 'POST /predict']
[PASS] GET /model-info returns model metadata: HTTP 200, parameters=4810, test_accuracy=0.9722
[PASS] GET /samples returns 10 labelled examples: HTTP 200, 10 samples
[PASS] POST /predict classifies the 10 samples: 10/10 correct
        true 0 -> predicted 0 (confidence 1.0)
        true 1 -> predicted 1 (confidence 0.9989)
        true 2 -> predicted 2 (confidence 1.0)
        true 3 -> predicted 3 (confidence 1.0)
        true 4 -> predicted 4 (confidence 1.0)
        true 5 -> predicted 5 (confidence 1.0)
        true 6 -> predicted 6 (confidence 0.9972)
        true 7 -> predicted 7 (confidence 1.0)
        true 8 -> predicted 8 (confidence 0.9951)
        true 9 -> predicted 9 (confidence 0.9837)
[PASS] POST /predict batch of 10 instances: HTTP 200, count=10, inference_ms=2.973
[PASS] Wrong input length is rejected with 400: HTTP 400 {"error": "Each image needs exactly 64 pixel values (flattened 8x8)."}
[PASS] Non-numeric pixels are rejected with 400: HTTP 400 {"error": "All pixel values must be numbers."}
[PASS] Out-of-range pixels are rejected with 400: HTTP 400 {"error": "Pixel values must be finite numbers between 0 and 16."}
[PASS] Invalid JSON is rejected with 400: HTTP 400 {"error": "Request body must be a JSON object with \"pixels\" or \"instances\"."}
[PASS] Unknown route returns JSON 404: HTTP 404 {"error": "Route not found."}
[PASS] GET on /predict returns JSON 405: HTTP 405 {"error": "Method not allowed for this route."}
[PASS] Latency of 50 sequential /predict calls measured: {"requests": 50, "mean_ms": 1.4, "median_ms": 1.29, "max_ms": 3.65}

13/13 checks passed
```

Figure 6: python3 test_api.py --url http://localhost:8000 against the running container.

What the screenshot shows

- All 13 checks pass ('13/13 checks passed').
- The 10 sample digits (0 to 9) are all classified correctly; the lowest confidence is 0.9837 for the digit 9.
- Wrong length, non-numeric, out-of-range and invalid-JSON inputs are rejected with HTTP 400; an unknown route returns JSON 404 and GET on /predict returns JSON 405.
- Latency of 50 sequential /predict calls: mean 1.4 ms, median 1.29 ms, maximum 3.65 ms; the batch of 10 took 2.973 ms of inference.

7.7 The API in a Web Browser

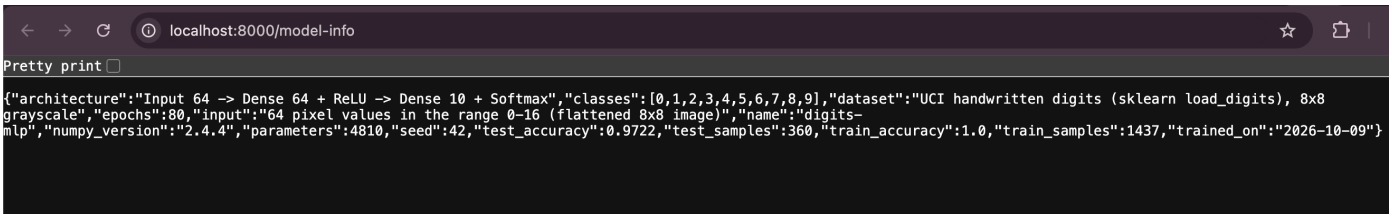


Figure 7: http://localhost:8000/model-info opened in the browser.

What the screenshot shows

- The same JSON as in Figure 4 is shown in the browser, so the container is reachable from the Mac through host port 8000.

7.8 Non-root User and Resource Usage

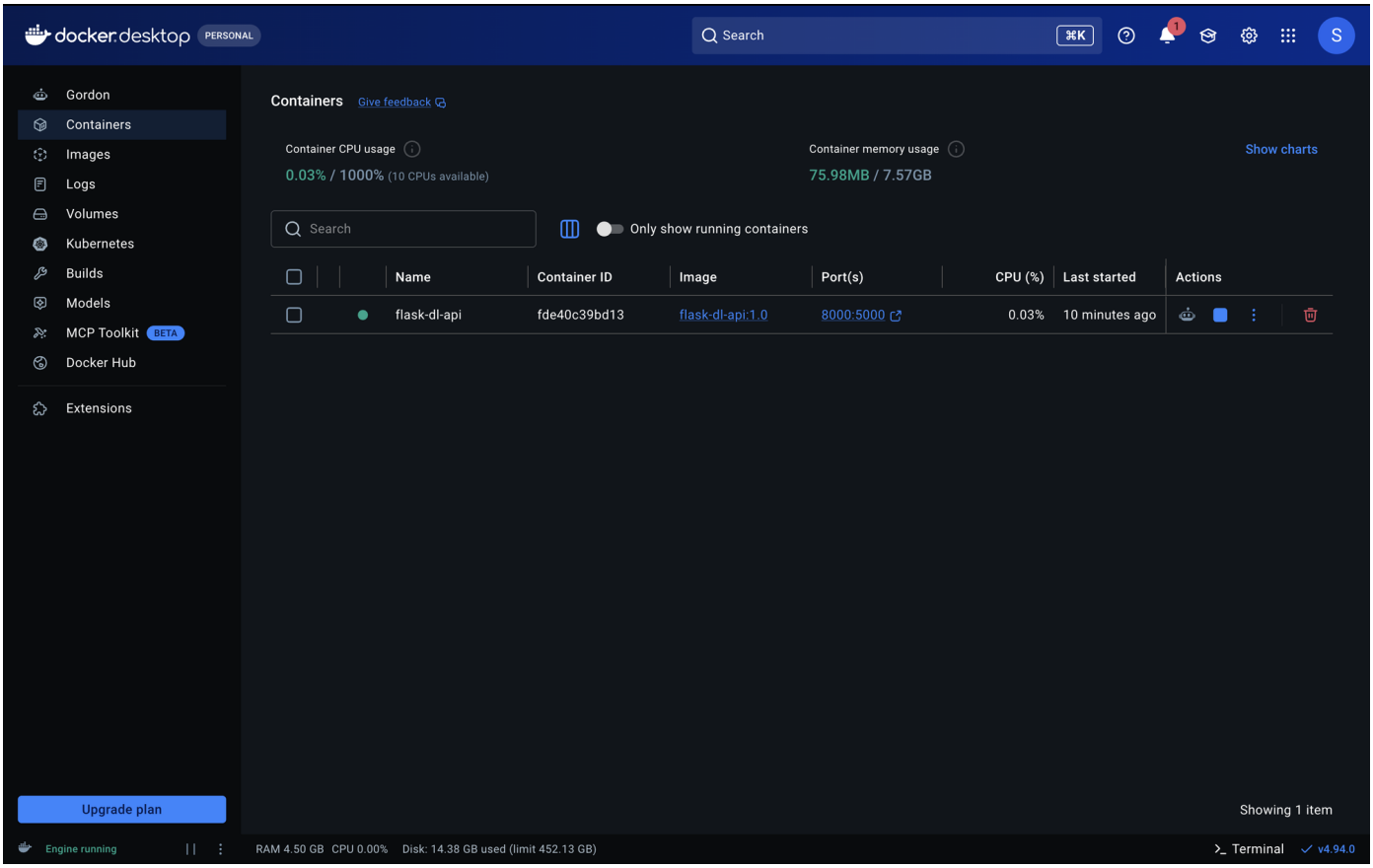
```
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task10 % docker exec flask-dl-api id
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task10 % docker stats --no-stream flask-dl-api
CONTAINER ID   NAME           CPU %     MEM USAGE / LIMIT    MEM %     NET I/O         BLOCK I/O   PIDS
fde40c39bd13   flask-dl-api   0.03%     74.9MiB / 7.748GiB   0.94%     70kB / 61.2kB   0B / 0B     22
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task10 %
```

Figure 8: docker exec flask-dl-api id and docker stats --no-stream flask-dl-api.

What the screenshot shows

- id: uid=10001(appuser) gid=10001(appuser) - the application does not run as root.
- docker stats: CPU 0.03 %, memory 74.9 MiB of 7.748 GiB (0.94 %), network 70 kB in and 61.2 kB out, block I/O 0 B, 22 processes.

7.9 Docker Desktop - Containers Tab



Enlarged detail of the same screen

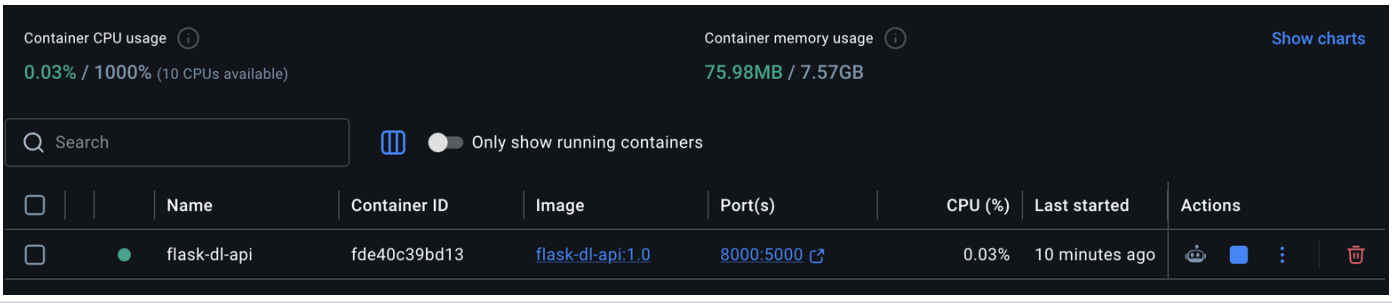


Figure 9: Docker Desktop, Containers tab, with the running container flask-dl-api. The lower image enlarges the table area of the same screen.

What the screenshot shows

- Row for flask-dl-api: container ID fde40c39bd13, image flask-dl-api:1.0, port 8000:5000, CPU 0.03 %, started 10 minutes ago, green status dot.
- Summary at the top: container CPU 0.03 % of 1000 % (10 CPUs available) and memory 75.98 MB of 7.57 GB; footer 'Engine running', Docker Desktop v4.94.0.

8. Results

The tables summarise the recorded runs. Values are taken from the command outputs in Section 6, the files `api_test_results.json` and `image_size_comparison.txt` and the screenshots of Section 7.

Metric	Value
Commands executed / succeeded	28 / 28 (100 %) in the recorded scripted run
Model	digits-mlp, 4,810 parameters, test accuracy 97.22 % (360 images), training accuracy 100 %
Image size, optimized	303,293,360 bytes (303.3 MB); content size 65.7 MB
Image size, baseline	1,752,970,368 bytes (1,753.0 MB); content size 443 MB
Size reduction	82.7 % (1,449.7 MB) in disk usage; 85.2 % in content size (65.7 MB against 443 MB)
Time until healthy	about 6 seconds (fourth check), also after docker restart
API test suite	13 of 13 checks passed (scripted run and manual run)
Sample predictions	10 of 10 digits correct; lowest confidence 0.9837 (digit 9)
Latency of /predict (50 calls)	scripted run: mean 1.36 ms, median 1.30 ms, max 2.38 ms; manual run: mean 1.4 ms, median 1.29 ms, max 3.65 ms
Container user	appuser (uid 10001); baseline: root (uid 0)
Idle memory (docker stats, step 27)	optimized 68.77 MiB, baseline 86.24 MiB
Training dependency in the image	scikit-learn not installed, NumPy installed

8.1 Results by approach area

Approach area	Steps	Succeeded
Verify Docker	1-2	2/2
Build the Images and Compare Their Size	3-8	6/6
Run the Container Locally	9-11	3/3
Verify API Functionality	12-16	5/5
Inspect the Container and Check the Optimizations	17-23	7/7
Compare with the Baseline Container	24-27	4/4
Cleanup	28	1/1

8.2 Optimized image compared with the baseline

Property	Baseline	Optimized
Dockerfile	`Dockerfile.baseline`	`Dockerfile`
Base image	python:3.12	python:3.12-slim
Build stages	1	2 (builder + runtime)
pip option	default (cache kept)	--no-cache-dir
Server process	Flask development server (python app.py)	gunicorn, 2 workers
Disk usage (docker images)	1.75 GB	303 MB
Content size	443 MB	65.7 MB
Container user	root (uid 0)	appuser (uid 10001)
Health status in docker ps	none	(healthy)
Idle memory / processes (step 27)	86.24 MiB / 10	68.77 MiB / 22

8.3 API test results (api_test_results.json)

Test	Result
GET /health returns 200 and status ok	PASS
GET / lists the endpoints	PASS
GET /model-info returns model metadata	PASS
GET /samples returns 10 labelled examples	PASS
POST /predict classifies the 10 samples	PASS
POST /predict batch of 10 instances	PASS
Wrong input length is rejected with 400	PASS
Non-numeric pixels are rejected with 400	PASS
Out-of-range pixels are rejected with 400	PASS
Invalid JSON is rejected with 400	PASS
Unknown route returns JSON 404	PASS
GET on /predict returns JSON 405	PASS
Latency of 50 sequential /predict calls measured	PASS

8.4 Sample predictions

True digit	Predicted	Confidence
0	0	1.0000
1	1	0.9989
2	2	1.0000
3	3	1.0000
4	4	1.0000
5	5	1.0000
6	6	0.9972
7	7	1.0000
8	8	0.9951
9	9	0.9837

9. Observations

- **The container works end to end.** The image builds, the container becomes healthy about 6 seconds after the start, all API routes answer through host port 8000, and 13 of 13 tests pass.
- **The optimizations are measurable.** The optimized image is 303.3 MB against 1,753.0 MB for the baseline (82.7 % smaller; 65.7 MB against 443 MB content size). The largest single layer of the optimized image is the 78.3 MB of installed packages, besides the base image layers.
- **Training code stays out of the image.** scikit-learn is not installed, /app contains only app.py and the 25 KB model file, and the API needs only NumPy for inference.
- **Security and operations improved.** The optimized container runs as appuser (uid 10001) while the baseline runs as root, has a HEALTHCHECK that Docker reports as (healthy), and recovers by itself after docker restart (healthy again at the fourth check).
- **The model is accurate and fast enough for a demo.** It classifies all 10 sample digits correctly (lowest confidence 0.9837), reaches 97.22 % on 360 held-out images, and a /predict call takes about 1.4 ms on average including the HTTP round trip (inference alone about 0.9 ms).
- **Input validation protects the API.** Wrong length, non-numeric values, out-of-range values and invalid JSON are answered with JSON errors and HTTP 400 instead of server errors.
- **Memory at idle is similar or lower.** In one idle snapshot the optimized container used 68.77 MiB (22 processes with gunicorn) and the baseline 86.24 MiB (10 processes with the development server).
- **A failed first run was recoverable.** The first run of the script lost the container steps; running it again produced the complete, successful log used in this report.

10. Known Limitations

- **Small demonstration model.** The network is a small MLP on 8x8 digit images (1,797 images). It reaches 100 % training accuracy against 97.22 % test accuracy, so it fits the training data more closely than unseen data; it is not a production model. No GPU is used.
- **Input format.** The API accepts only flattened 64-value pixel vectors, not image files.
- **Limited load testing.** Latency was measured with 50 sequential requests from the same machine; concurrent load and long-running behaviour were not tested.
- **Architecture-specific numbers.** The images are arm64 (Apple Silicon); sizes and IDs differ on other machines, and base-image tags such as python:3.12-slim change over time.
- **Single snapshots.** The memory comparison is one idle measurement, and build durations are not compared because layers were cached.
- **No security hardening beyond the basics.** There is no authentication or HTTPS, and no image scanning; the image was not pushed to a registry and no orchestration (Compose or Kubernetes) was used.
- **Screenshots from a second run.** The IDs in the screenshots differ from the log (see Section 7); the Docker Desktop Containers tab is the only graphical screenshot.

11. Conclusion

A Flask-based deep-learning API (a NumPy neural-network digit classifier with 97.22 % test accuracy) was packaged into a Docker container. A multi-stage Dockerfile with a slim base image, no pip cache, pinned dependencies, a .dockerignore file, a non-root user and a health check produced the image flask-dl-api:1.0 of 303.3 MB, which is 82.7 % smaller than the un-optimized baseline of 1,753.0 MB. The container was run locally on host port 8000, became healthy after about 6 seconds, passed 13 of 13 API checks (including 10 of 10 sample digits and the rejection of invalid input) and recovered after a restart. All 28 commands of the recorded run succeeded. Together with the Dockerfile, the image, the command log, the screenshots and the observations above, this report documents the containerization and deployment of the Flask deep-learning API.